

Aula 21: Modelos de Difusão

Denoising Diffusion Probabilistic Model (DDPM) e Score-Based Generative Modeling

Disciplina de Aprendizado Profundo

Objetivos da Aula

- Introduzir o paradigma de geração por difusão reversa.
- Entender o processo de destruição progressiva da informação (forward diffusion).
- Derivar a loss principal dos DDPMs (ϵ -prediction).
- Compreender sampling e guidance.
- Conectar DDPM a score-based models derivados de SDEs.
- Esboçar um DDPM mínimo (para o notebook da aula prática).

Panorama dos Modelos Generativos Modernos

- GANs: treinamento adversarial, instabilidade, *mode collapse*.
- VAEs: ELBO estável, mas gerações frequentemente desfocadas.
- Normalizing Flows: bijeções invertíveis, custo alto em alta dimensão.
- **Modelos de Difusão**: estáveis, escaláveis, alta qualidade.



A Ideia Central: Destruir + Reconstruir

Forward diffusion

Adicionar ruído Gaussiano passo a passo até aproximar uma distribuição isotrópica:

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_T \sim \mathcal{N}(0, I).$$

Reverse diffusion

Treinar uma rede neural para reverter esse processo:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

Intuição Probabilística

- Aproximamos uma distribuição complexa (imagens) via uma cadeia de Markov.
- O forward é fixo, conhecido e simples.
- O reverse é desconhecido e precisa ser aprendido.
- A rede precisa aprender “como desfazer o ruído”.

Por que isso funciona tão bem?

- O espaço de ruído é muito mais simples de modelar.
- Cada passo de difusão é local e fácil de aprender.
- O treinamento é estável — não há adversário (ao contrário de GANs).
- Permite condicionamento flexível (texto, pose, rascunho, áudio, etc.).

Construindo o Processo Forward

Definimos uma sequência de ruídos independentes:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right),$$

onde β_t controla a intensidade do ruído.

- β_t pequeno: pouco ruído por passo.
- β_t grande: destruição mais agressiva.
- β_t segue um *schedule* (linear, cosine, etc.).

Definindo α_t e $\bar{\alpha}_t$

Convenções importantes:

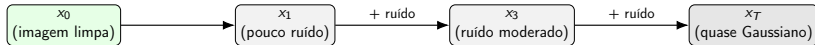
$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Com isso, temos uma expressão fechada:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Isso nos permite amostrar qualquer x_t diretamente a partir de x_0 .

Visualização da Difusão Forward (Esquemática)



Interpretação Geométrica

- A difusão “alisa” a geometria do espaço de imagens.
- O dado original se aproxima gradualmente de ruído isotrópico.
- Esse processo é reversível em expectativa.

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon.$$

Objetivo de um Passo de Difusão

Queremos definir uma transição de Markov

$$q(x_t \mid x_{t-1})$$

que satisfaça:

- Seja **simples de amostrar** (para construirmos o forward).
- Seja **Gaussiana** (fechada, bem comportada).
- Aumente gradualmente o ruído na amostra:

$$\mathbb{E}[x_t \mid x_{t-1}] \propto x_{t-1}, \quad \text{Var}(x_t \mid x_{t-1}) \text{ controlada por } \beta_t.$$

Uma escolha natural: **combinar linearmente** o sinal anterior com ruído Gaussiano.

Atualização Linear com Ruído Gaussiano

Vamos assumir uma forma *gerativa* para x_t :

$$x_t = a_t x_{t-1} + b_t \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I).$$

Condições desejadas:

- A média condicional é proporcional a x_{t-1} :

$$\mathbb{E}[x_t \mid x_{t-1}] = a_t x_{t-1}.$$

- A variância condicional é isotrópica:

$$\text{Var}(x_t \mid x_{t-1}) = b_t^2 I.$$

Logo, **para quaisquer** a_t, b_t , já temos que

$$q(x_t \mid x_{t-1}) = \mathcal{N}(a_t x_{t-1}, b_t^2 I).$$

Escolha Específica de a_t e b_t

Nos modelos de difusão, escolhemos uma sequência de **intensidades de ruído** (β_t).

Uma parametrização conveniente é:

$$a_t = \sqrt{1 - \beta_t}, \quad b_t = \sqrt{\beta_t}.$$

Com isso:

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t,$$

e, pelo fato de soma de Gaussianas ser Gaussiana:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$

Esta é exatamente a fórmula usada:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$

Por que essa escolha é conveniente?

Com

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t,$$

podemos compor vários passos:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s),$$

para algum $\varepsilon \sim \mathcal{N}(0, I)$.

Vantagens:

- **Fechamos** a forma de $q(x_t \mid x_0)$.
- Podemos amostrar x_t **diretamente** a partir de x_0 em uma única equação.
- Simplifica tanto a teoria quanto a implementação (função `q_sample` no notebook).

Resumindo a Derivação

- 1 Partimos de uma forma linear com ruído: $x_t = a_t x_{t-1} + b_t \varepsilon_t$.
- 2 Isso implica automaticamente: $q(x_t | x_{t-1}) = \mathcal{N}(a_t x_{t-1}, b_t^2 I)$.
- 3 Escolhemos $a_t = \sqrt{1 - \beta_t}$, $b_t = \sqrt{\beta_t}$ para controlar diretamente o “tamanho” do passo de ruído.
- 4 Daí obtemos:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I),$$

que é a fórmula usada em DDPM.

Nosso Objetivo

Queremos modelar a transição reversa:

$$p_{\theta}(x_{t-1} \mid x_t).$$

Assumimos forma Gaussiana:

$$p_{\theta}(x_{t-1} \mid x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \sigma_t^2 I),$$

onde $\mu_{\theta}(x_t, t)$ é parametrizada por uma rede neural.

O Problema Central

Não sabemos $\mu_\theta(x_t, t)$ diretamente.

Mas sabemos:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon.$$

Se a rede aprender $\varepsilon_\theta(x_t, t)$ tal que:

$$\varepsilon_\theta(x_t, t) \approx \varepsilon,$$

então podemos reconstruir $\hat{x}_0$ e, a partir dela, obter o reverse.

Parametrização ϵ -prediction

A rede neural prediz o ruído:

$$\varepsilon_{\theta}(x_t, t).$$

Loss principal:

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2] .$$

- Simples, estável e muito eficaz.
- É a loss que usamos na prática em DDPMs.

Reconstrução de x_0

Da expressão:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon,$$

obtemos o estimador:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

A partir de $\hat{x}_0$, podemos derivar $\mu_\theta(x_t, t)$.

A Loss Completa (visão ELBO)

A loss total (ELBO) pode ser escrita como:

$$L_{\text{ELBO}} = L_0 + L_1 + \cdots + L_T,$$

onde tipicamente:

$$L_t \propto \mathbb{E} [\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2].$$

Na prática, usamos L_{simple} (com pesos adequados) como aproximação.

Relação com Score Matching

Para o forward:

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I),$$

temos:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{1}{(1 - \bar{\alpha}_t)}(x_t - \sqrt{\bar{\alpha}_t}x_0) \propto -\varepsilon.$$

Logo:

$$\varepsilon_\theta(x_t, t) \approx -\sigma_t \cdot s_\theta(x_t, t),$$

onde s_θ modela o *score*.

Score-Based SDEs (Super Resumo)

Forward:

$$dx_t = f(x_t, t) dt + g(t) dW_t.$$

Reverse SDE:

$$dx_t = [f(x_t, t) - g(t)^2 s_\theta(x_t, t)] dt + g(t) dW_t.$$

Sampling $\Rightarrow$ integrar a SDE reversa usando um solver numérico.

Vamos começar com um Denoising Autoencoder (DAE)

O DAE é o ancestral conceitual dos modelos de difusão.

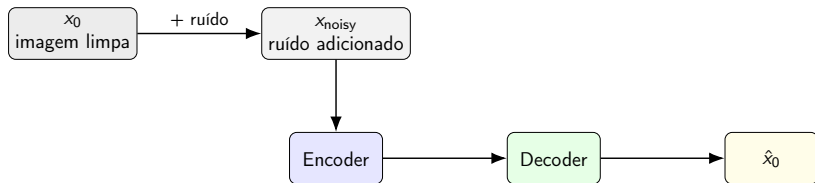
- Recebe uma versão ruidosa da imagem: x_{noisy} .
- Prediz uma versão limpa $\hat{x}_0$.
- Aprende a “desfazer” uma perturbação estocástica.

Isso é exatamente o que precisamos em difusão:

dado x_t (ruidoso), prever algo sobre x_0 .

O DAE é, portanto, a **primeira aproximação** ao processo reverso dos DDPMs.

Esquema de um Denoising Autoencoder



$$L = \|x_0 - \hat{x}_0\|^2.$$

Relação entre DAE e Score Matching

Quando adicionamos ruído Gaussiano:

$$x_{\text{noisy}} = x_0 + \sigma \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

Vincent (2011) provou que o DAE aprende:

$$\hat{x}_0(x) - x \propto \nabla_x \log p(x),$$

ou seja, o **score** da distribuição!

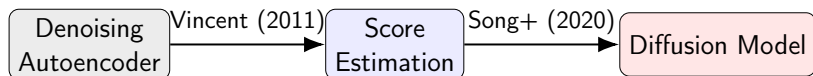
Logo:

DAE $\implies$ aproximação do score.

E modelos de difusão são exatamente:

score models + SDE reversa.

Conexão Intuitiva: DAE $\rightarrow$ Difusão



- DAE aprende a remover ruído fixo.
- Difusão usa ruídos variáveis $t = 1, \dots, T$.
- Difusão aprende múltiplos DAEs encadeados.

Um DAE é um DDPM com $T = 1$

Um DDPM geral define:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon.$$

Se escolhermos um único passo ($T = 1$):

$$x_1 = \sqrt{1 - \beta}x_0 + \sqrt{\beta}\epsilon.$$

Treinamos a rede para prever diretamente x_0 :

$$\hat{x}_0 = f_{\theta}(x_1),$$

que é exatamente um **denoising autoencoder**.

Implementação Rápida de um DAE para MNIST

Estrutura mínima no notebook:

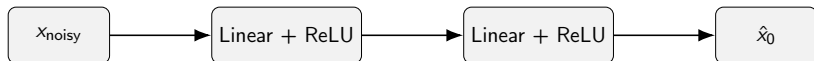
- 1 Carregar MNIST normalizado para $[-1, 1]$.
- 2 Amostrar ruído Gaussiano:

$$x_{\text{noisy}} = x_0 + \sigma \varepsilon.$$

- 3 Passar pelo MLP ou CNN.
- 4 Minimizar:

$$L = \|x_0 - \hat{x}_0\|^2.$$

Arquitetura do DAE (MLP Simples)



- Para MNIST: achatar $28 \times 28 \rightarrow 784$.
- Camadas típicas: $784 \rightarrow 512 \rightarrow 512 \rightarrow 784$.
- Reconstrução reshape para 28×28 .

Arquitetura do DAE (CNN)



- CNN funciona muito melhor que MLP para imagens.
- Estrutura similar à UNet, mas sem skip connections.

Exercício no Notebook: Implementar o DAE

No Colab os estudantes devem:

- 1 Implementar `add_noise(x, sigma)`.
- 2 Implementar o forward da rede (MLP ou CNN).
- 3 Treinar por algumas épocas.
- 4 Visualizar pares: $(x_0, x_{\text{noisy}}, \hat{x}_0)$.

Conexão esperada:

$$\hat{x}_0 - x_{\text{noisy}} \approx \text{direo de limpeza do score.}$$

Resumo da Seção: DAE $\rightarrow$ Difusão

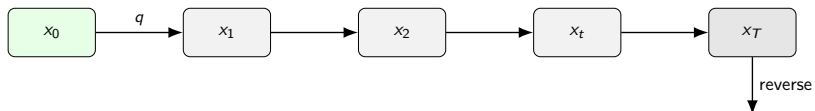
- O DAE treina um único passo de “desfazer ruído”.
- O DDPM treina milhares de DAEs, um por timestep.
- Ambos estimam o **score** da distribuição em diferentes níveis de ruído.

Agora que entendemos o DAE, vamos generalizá-lo para T passos: difusão.

Comparação: DDPM vs Score-SDE

- DDPM: processo discreto de Markov, Gaussiano.
- SDE-based: processo contínuo, generaliza DDPM.
- Ambos usam score/ ϵ -prediction.
- Ambos produzem imagens de altíssima qualidade (*SOTA*).

Diagrama Geral de um DDPM



Rede Neural do Reverse

- Entrada: (x_t, t) .
- Saída: $\varepsilon_\theta(x_t, t)$.
- Arquitetura típica:
 - UNet (com convoluções 2D).
 - Blocos residuais (*ResBlocks*).
 - *Skip connections* entre escalas.
 - Embedding temporal sinusoidal.

Até Aqui: Resumo

- Forward diffusion: adiciona ruído com schedule β_t .
- Reverse diffusion: é aprendido como Gaussiana reversa.
- Loss principal: prever o ruído verdadeiro (ϵ -prediction).
- Conexão forte com score matching e SDEs reversas.

Próximo: arquiteturas (*UNet*) e *schedulers*.

UNet: Arquitetura Essencial

- Arquitetura originalmente para segmentação, mas ideal para difusão.
- Combina convoluções, downsampling e upsampling.
- Mantém alta resolução via *skip connections*.

UNet para Diffusion (visão prática)

- Entrada: imagem ruidosa x_t .
- Entrada adicional: embedding temporal t .
- Saída: previsão do ruído $\varepsilon_\theta(x_t, t)$.
- Blocos principais:
 - ConvBlocks + ResBlocks.
 - Downsample / Upsample.
 - *Skip connections* entre níveis.

Embedding Temporal

Cada timestep t deve ser representado de forma contínua:

$$\text{emb}(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \dots, \sin(\omega_d t), \cos(\omega_d t)].$$

- Mesmo princípio do positional encoding em Transformers.
- Permite que a rede saiba “quão ruidosa” está a entrada.

Scheduler de Ruído

- O scheduler determina os valores de β_t .
- Controla quão rápido destruimos a imagem.

$$\beta_1, \beta_2, \dots, \beta_T.$$

Tipos comuns:

- Linear (DDPM original).
- Cosine (mais usado atualmente).
- Sigmoid, Quadrático.

Scheduler Cosine (Ideia)

A ideia é escolher $\bar{\alpha}_t$ de forma suave em t :

$$\bar{\alpha}_t = \frac{\cos^2\left(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\right)}{\cos^2\left(\frac{s}{1+s} \cdot \frac{\pi}{2}\right)},$$

onde s é um parâmetro pequeno.

- Distribui o ruído de maneira mais adequada ao longo do horizonte.
- Usado em modelos modernos (p.ex. Improved DDPM).

Treinamento DDPM: Overview

Para cada batch:

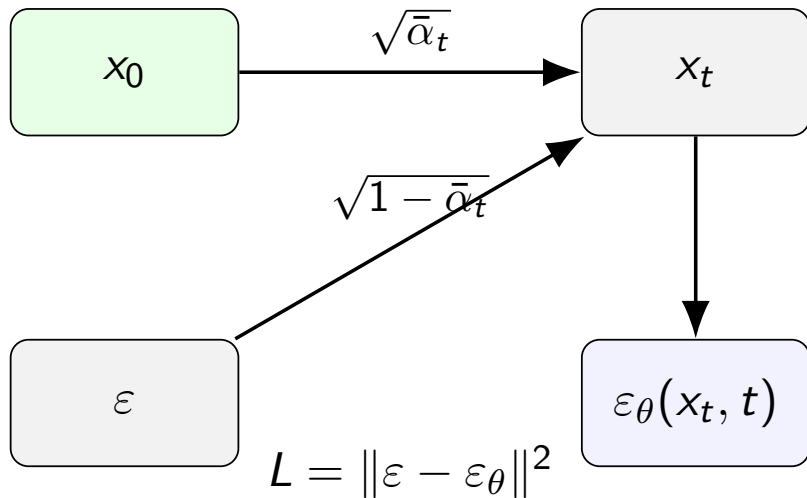
- 1 Amostrar x_0 do dataset.
- 2 Amostrar $t \sim \text{Uniform}\{1, \dots, T\}$.
- 3 Amostrar ruído $\varepsilon \sim \mathcal{N}(0, I)$.
- 4 Criar x_t :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon.$$

- 5 Treinar a rede para prever ε :

$$L = \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2.$$

Treinamento: Fluxo Computacional



Sampling Reverso (DDPM)

Usamos a transição:

$$p_{\theta}(x_{t-1} \mid x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \sigma_t^2 I).$$

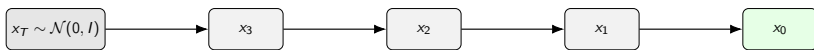
Média:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(x_t, t) \right).$$

Sampling:

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

Sampling: Diagrama Discreto



Observações Importantes sobre Sampling

- Sampling é mais caro que o forward de uma única imagem.
- Precisamos de muitos steps (p.ex. $T = 1000$) no DDPM original.
- Podemos reduzir steps via DDIM ou outros samplers.
- Sampling é facilmente paralelizável por batch.

DDIM (Deterministic Diffusion)

Versão determinística de sampling:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon_{\theta}(x_t, t),$$

onde:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

- Permite sampling mais rápido com menos steps.
- Mantém boa qualidade visual.

Interpretação de DDIM

- DDPM: sampling totalmente estocástico (cada passo adiciona ruído).
- DDIM: sampling *quase determinístico*, ruído opcional.
- Permite interpolar no espaço de difusão: mistura suave de amostras.

Classifier-Free Guidance (CFG)

Ideia: gerar imagens condicionais sem treinar dois modelos distintos.
Definimos:

$$\hat{\varepsilon}_{\theta} = (1 + w) \varepsilon_{\theta}(x_t, t, c) - w \varepsilon_{\theta}(x_t, t, \emptyset),$$

onde:

- c = condição (texto, classe, etc.);
- $\emptyset$ = condição nula (sem prompt);
- w = força do *guidance*.

Intuição do CFG

- Durante o treino, às vezes usamos o prompt c , às vezes deixamos vazio.
- O modelo aprende a “entender” as duas distribuições:
 - condicional $p(x|c)$;
 - não condicional $p(x)$.
- No sampling, combinamos as duas previsões para empurrar a amostra em direção ao prompt.

Efeito do Guidance (Esquemático)

Sem guidance ($w \approx 0$)

Guidance forte ($w \gg 1$)

mais diversidade, menos aderência ao prompt menos diversidade, maior fidelidade ao prompt

Resumo até Aqui

- UNet + embedding temporal $\Rightarrow$ arquitetura principal de difusão.
- Forward diffusion: fixo, adiciona ruído Gaussiano com schedule.
- Reverse diffusion: aprendido via ϵ -prediction.
- Sampling: DDPM (estocástico) ou DDIM (parcialmente determinístico).
- Classifier-Free Guidance: melhora condicionamento sem modelo extra.

Próximo: Implementação (Visão de Código)

Vamos especificar:

- Dataset (p.ex. MNIST ou CIFAR-mini).
- Scheduler linear/cosine para β_t .
- Função `q_sample(x0, t)` para obter x_t .
- Treino: loop de otimização com loss MSE no ruído.
- Sampling DDPM/DDIM passo a passo.

Conexão com Score Matching (Relembrando)

A difusão se conecta a score matching:

$$s_{\theta}(x_t, t) = \nabla_{x_t} \log p_t(x_t),$$

e:

$$\varepsilon_{\theta}(x_t, t) \propto -s_{\theta}(x_t, t).$$

Modelos de difusão podem ser vistos como *score models* discretizados com ruído controlado.

Checklist do que já entendemos

- Forward diffusion (q).
- Reverse diffusion (p_θ).
- Loss de ϵ -prediction.
- Scheduler de ruído.
- Sampling DDPM e DDIM.
- Ideia de guidance (CFG).

Na próxima parte: foco total em implementação e experimentos.

Parte Prática: Implementando um DDPM Mínimo

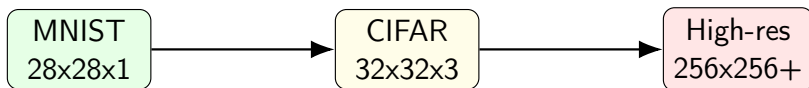
Nesta parte vamos focar em aspectos concretos:

- Dataset simples (MNIST ou CIFAR mini).
- Scheduler de ruído e função `q_sample`.
- Arquitetura da rede (UNet mini).
- Detalhes de treino (otimizador, normalização, etc.).
- Sampling e visualização de amostras.



Escolha de Dataset e Resolução

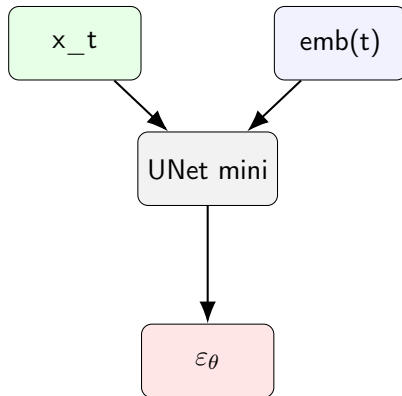
- **Para começar:** MNIST (28x28, 1 canal) é ideal.
- Próximo passo: CIFAR-10 (32x32, 3 canais).
- Resolução maior (256x256, 512x512) exige:
 - UNet maior;
 - Mais memória;
 - Tempo de treino muito maior.



UNet Mini vs UNet Completa

- **UNet mini (para MNIST):**
 - Poucos canais (ex.: 32, 64).
 - 2 ou 3 níveis de down/up-sampling.
- **UNet completa (stable diffusion):**
 - Muitos canais (ex.: 320, 640, 1280).
 - Vários níveis + atenção.
 - Pesos enormes (centenas de milhões).

Esqueleto de Código da Rede



- A embedding temporal é injetada em vários blocos da UNet.

Scheduler e Função `q_sample`

- No código, implementamos:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon.$$

- Função típica no notebook:

`q_sample(x0, t, noise)  $\Rightarrow$   $x_t$ .`

- Pré-computamos vetores α_t , $\bar{\alpha}_t$ (tensores 1D).
- Indexamos por t durante o treino.

Loop de Treino: Estrutura Geral

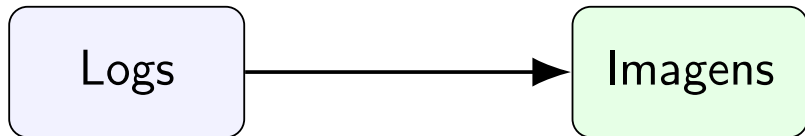
- Para cada batch x_0 :
 - 1 Amostra-se timestep t .
 - 2 Gera-se ruído noise .
 - 3 Computa-se $x_t = q_sample(x_0, t, \text{noise})$.
 - 4 Prediz-se $\text{noise_pred} = \text{model}(x_t, t)$.
 - 5 Loss: MSE entre noise e noise_pred .
- **Otimizador:** Adam ou AdamW.

Questões Práticas de Treino

- **Batch size:** limitado por memória (começar pequeno).
- **Learning rate:** típico 10^{-4} a 10^{-3} .
- **Gradient clipping:** pode ajudar em instabilidades.
- **EMA de pesos:** manter um modelo suavizado para sampling.
- **Mixed precision:** importante em alta resolução.

Monitorando o Treino

- Loss total (MSE no ruído).
- Loss média por timestep (opcional).
- Visualizar amostras parciais:
 - Reconstruções a partir de x_T .
 - Trajetórias $x_T \rightarrow x_0$ para um batch pequeno.



Debugando o Forward Diffusion

- Pegue uma única imagem x_0 .
- Gere x_t para alguns valores de t (pequeno, médio, grande).
- Verifique se:
 - t pequeno: imagem pouco ruidosa.
 - t próximo de T : quase puro ruído.
- Esse teste ajuda a detectar erros em $\bar{\alpha}_t$ e q_{sample} .

Debugando o Reverse Diffusion

- Use um modelo *undertrained* (poucas épocas).
- Execute o sampling completo $x_T \rightarrow x_0$.
- Verifique se:
 - As imagens não explodem (NaN, saturação extrema).
 - Há algum padrão emergindo com o tempo de treino.
- Se nada aparece após muitas épocas, verifique:
 - Embedding temporal t ;
 - Indexação do scheduler.

Notebook no Colab: Organização

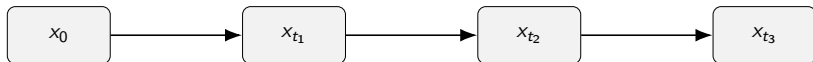
No Colab teremos um notebook com a seguinte estrutura:

- 1 **Setup:** imports, configuração de dispositivo.
- 2 **Dataset:** carregamento de MNIST.
- 3 **Scheduler:** cálculo de α_t , $\bar{\alpha}_t$.
- 4 **Rede:** UNet mini + embedding temporal.
- 5 **Treino:** loop de otimização com MSE no ruído.
- 6 **Sampling:** função para gerar amostras a partir de x_T .

Exercício 1 — Forward Diffusion no Notebook

No Colab:

- Completar a função `q_sample(x0, t, noise)`.
- Plotar visualmente x_t para diferentes t .



Exercício 2 — Treino da Rede

No Colab:

- Completar o *forward* do modelo (`model(x_t, t)`).
- Implementar a loss MSE entre ruído verdadeiro e predito.
- Treinar por algumas épocas.
- Logar a loss ao longo do tempo.

Exercício 3 — Sampling e Visualização

No Colab:

- Implementar a rotina de sampling DDPM (com ruído).
- Visualizar amostras geradas a partir de $x_T \sim \mathcal{N}(0, I)$.
- (Opcional) Implementar sampling DDIM e comparar.

Resumo da Parte Prática

- Vimos as principais decisões de implementação em DDPM.
- Estruturamos o código em termos de:
 - scheduler, `q_sample`, UNet, treino, sampling.
- No Colab, você terá um notebook com **TODOs** para completar.

Perguntas antes de irmos para o notebook?